

AIMS System - Detailed Class Design Documentation

Name: Vũ Hải Đăng

Student ID: 20225962

1. Design Class: UpdateCartController

Table 1. Example of operation design

#	Name	Return type	Description (purpose)
1	requestToUpdateCart	boolean	Initiates cart update process
2	updateQuantity	vold	Updates the quantity of a specific product in the cart.
3	removeProductFromCart	vold	Removes a specific product from the cart.

Parameter:

- **requestToUpdateCart(productId: int, quantity: int, cartID: int): vold**
 - **productId:** The unique Identifier of the product to update in the cart.
 - **cartID:** The shopping cart that contains the products to update quantity.
 - **quantity:** The new quantity of the product.
 - **Return:** None
- **updateQuantity(productId: int, quantity: int, cartID: int): vold**
 - **productId:** The unique Identifier of the product whose quantity needs to be updated.
 - **quantity:** The updated quantity to be set for the product.
 - **cartID:** The shopping cart that contains the products to update quantity.
 - **Return:** None
- **removeProductFromCart(productId: int, cartID: int): vold**
 - **productId:** The unique Identifier of the product to be removed from the cart.
 - **cartID:** The shopping cart that contains the products to update quantity.

Exception:

- **requestToUpdateCart(productId: int, quantity: int, cartID: int): vold**
 - **ProductNotFoundException:** If the specified product does not exist.
- **updateQuantity(productId: int, quantity: int, cartID: int): vold**
 - **InsufficientStockException:** If the requested quantity exceeds available stock.

Method:

How to Use Parameters/Attributes

- **requestToUpdateCart(productId: int, quantity: int, cartID: int): vold**
 - **Purpose:** This method initiates the cart update process when a user modifies the quantity or removes an item from the cart.
 - **Usage:**
 - Retrieve the product using productId. If the product is not found, throw ProductNotFoundException.
 - Update the cart.
- **updateQuantity(productId: int, quantity: int, cartID: int): vold**
 - **Purpose:** Updates the quantity of a specific product in the shopping cart.
 - **Usage:** Call inslde **requestToUpdateCart(productId: int, quantity: int, cartID: int): vold**
 - Retrieve the product from currentCart using productId.
 - Valldate the quantity (e.g., ensure it's a positive integer).
 - Update the product quantity in currentCart.If the requested quantity exceeds available stock, throw **InsufficientStockException**
- **removeProductFromCart(productId: int, cartID: int): vold**
 - **Purpose:** Removes a product from the shopping cart based on its unique Identifier.
 - **Usage:** Call inslde **requestToUpdateCart(productId: int, quantity: int, cartID: int): vold**
 - Retrieve the product from currentCart using productId.
 - Remove the product from currentCart.
 - Update the cart state accordingly.

State:

- **Checking Stock** – Verifying the stock quantity of the product.
- **Available** – The requested quantity is in stock (stock > requested quantity).
- **Not Available** – Insufficient stock (stock < requested quantity).
- **Error: Product Not Found** – The product ID is invalid (ProductNotFoundException).

2. Design Class: Product

Table 1. Attribute design

#	Name	Data type	Default value	Description
1	productID	int	null	Unique identifier for the product
2	title	String	null	Name of the product
3	category	String	null	Product category (Book, CD, DVD, or LP record)
4	value	float	0.0	Base price of the product before VAT
5	price	float	0.0	Selling price of the product
6	quantity	int	0	Available quantity in stock
7	description	String	null	Product description
8	imageUrl	String	null	The URL for the image of the product item
9	barCode	String	null	Unique barcode for the product
10	warehouseEntryDate	Date	null	Date when the product entered the warehouse
11	dimensions	String	null	Physical dimensions of the product (e.g., 10x20x5 cm)
12	weight	float	null	Weight of the product in kilograms
13	warehouseProvince	String	null	province of warehouse storing the product
14	warehouseDistrict	String	null	district of warehouse storing the product
15	warehouseAddress	String	null	detailed address of warehouse storing the product

Table 2. Operation design

#	Name	Return type	Description (purpose)
1	checkProductAvailability	boolean	Checks if the product has sufficient quantity available
2	updateProduct	void	Updates product information in the system
3	checkEligibilityOfRushDelivery	boolean	Checks if product is eligible for rush delivery
4	createProduct	void	Create a new product in the system
5	searchProduct	void	Search for a specific product attributes

Parameter:

- **checkProductAvailability(quantity: int): Boolean**
 - **quantity:** An integer representing the number of products to check if the available inventory is sufficient.
 - **Return:** true if the available inventory is sufficient, otherwise false.
- **updateProduct(productID: int): void**
 - **productID:** productID must exist
- **searchProduct(searchCriteria: Map<String, Object>): void**
 - **searchCriteria:** A map containing search criteria (field names and values)

Exception:

- **updateProduct(productID: int): void**
 - **ProductNotFoundException:** If no product with the specified productID is found.
 - **ExceedUpdateTimesException:** If update more than 30 products a day
 - **ExceedUpdatePriceTimesPerDayException:** If update price more than twice a day

Method:

How to use parameters/attributes:

- **checkProductAvailability(quantity: int): Boolean**
 - **Purpose:** This method checks whether the available inventory is sufficient for the requested quantity of a product.
 - **Usage:** If the available stock is at least equal to the requested quantity, the method returns true; otherwise, it returns false.
- **updateProduct(productID: int): void**
 - **Purpose:** This method updates the details of an existing product based on the provided information.
 - **Usage:** If the product exists, the method updates attributes such as name, price, quantity, etc
- **createProduct(): void**
 - **Purpose:** This method creates a new product with the provided information.
 - **Usage:** If all required attributes are provided and valid, the product is successfully created and added to the inventory.
- **checkEligibilityOfRushDelivery(address: String): Boolean**
 - **Purpose:** to check rush delivery support
 - **Usage:** compare ‘address’ parameter (from delivery information) and address of product (based on information provided by Product Manager)
- **searchProduct(searchCriteria: Map<String, Object>): void**
 - **Purpose:** Searches for products matching specific criteria.
 - **Usage:** Used to find products based on various attributes.
- **Processing:**
 - Constructs a database query based on the search criteria
 - Executes the query
 - Returns matching products

State:

- If stock > requested quantity → **Available.**
- If stock < requested quantity → **Not Available.**
- If product_id is invalid → ProductNotFoundException is raised.

3. Design Class: Cart

Table 1. Attribute design

#	Name	Data type	Default value	Description
1	cartItemsList	List<CartItem>	empty	List of CartItem in the cart
2	cartID	float	0.0	Unique identifier for the cart

Table 2. Operation design

#	Name	Return type	Description (purpose)
1	calculateTotalPrice	float	Calculates and returns the total price
2	empty	void	Empties the cart
3	checkEligibilityOfRushDeliveryOfChosenItems	boolean	Checks if all items are eligible for rush delivery
4	checkAvailabilityOfChosenItems	boolean	Checks if all items have sufficient inventory
5	isEmpty	boolean	Checks if the cart is empty

Parameter:

- **checkEligibilityOfRushDeliveryOfChosenItems(province: String, district: String): boolean**
 - **province:** The name of the province where the delivery is requested.
 - **district:** The name of the district within the province.
 - **Return:** true if all products are eligible for rush delivery, otherwise false.

Exception: None

Method:

How to use parameters/attributes:

- **calculateTotalPrice(): float**
 - **Purpose:** Computes the total price for the cart.
 - **Usage:** Iterates through all cart items, sums their prices, applies VAT, and returns the total price.
- **empty(): void**
 - **Purpose:** Empties the cart by removing all items.
 - **Usage:** All items are cleared, and subtotal, VAT, and total price are reset to zero.
- **checkEligibilityOfRushDeliveryOfChosenItems(province: String, district: String): boolean**
 - **Purpose:** Determines if all products in the cart are eligible for rush delivery.
 - **Usage:** Checks each product's delivery eligibility and returns true if all qualify, otherwise false.
- **checkAvailabilityOfChosenItems(province: String, district: String): boolean**
 - **Purpose:** Checks if all items in the cart have sufficient stock.
 - **Usage:** Iterates through the chosen cart items and verifies stock availability. Returns true if all items are available, otherwise false.
- **isEmpty(): boolean**
 - **Purpose:** Checks if the cart is empty.
 - **Usage:** Returns true if there are no items in the cart, otherwise false.

State:

- If cartItemsList is empty → **Empty**
- If cartItemsList contains at least one item → **Active**
- If checkout() is called and payment is not completed → **CheckedOut**
- If payment is successful → **Paid**
- If cart is emptied (either manually or after order completion) → **Empty**

4. Design Class: CartItem

Table 1. Attribute design

#	Name	Data type	Default value	Description
1	productId	int	null	ID of the product in the cart
2	quantity	int	0	Quantity of the product
3	cartID	int	null	Unique identifier for the cart
4	cartItemID	int	null	Unique identifier for the cart item

Table 2. Operation design

#	Name	Return type	Description (purpose)
1	changeQuantityOfProducts.	void	Updates the quantity and recalculates subtotal
2	addProductToCart	void	Adds a product to the cart with specified quantity

Parameter:

- **addProductToCart(product: Product, quantity: int): voids**
 - **product:** A Product object representing the product to add.
 - **quantity:** An integer representing the number of products to add.
 - **Return:** None.

- **changeQuantityofProduct(product: Product, quantity: int): void**

- **product:** A Product object representing the product to update.
- **quantity:** An integer representing the new quantity.
- **Return:** None.

Exception:

- **addProductToCart(productID: int, quantity: int): void**
 - **ProductNotFoundException:** Raised if the specified product does not exist in the system.
 - **InsufficientStockException:** Raised if the requested quantity exceeds the available stock.
- **changeQuantityOfProduct(productID: int, quantity: int): void**
 - **ProductNotFoundException:** Raised if the product is not found in the cart.
 - **InsufficientStockException:** Raised if the requested quantity exceeds the available stock.

Method:

How to use parameters/attributes:

- **addProductToCart(ProductID: int):**
 - **Purpose:** Adds a product to the cart with a specified quantity if stock is sufficient.
 - **Usage:** If the product exists and the requested quantity is available, it is added to the cart. Otherwise, InsufficientStockException is raised.
- **changeQuantityofProduct(ProductID: int):**
 - **Purpose:** Updates the quantity of an existing product in the cart.
 - **Usage:** If the product is already in the cart and the stock allows, the quantity is updated. Otherwise, InsufficientStockException is raised.

State

- **Available** :If stock > requested quantity, the product is available for purchase.
- **Not Available** :If stock < requested quantity, the requested quantity cannot be fulfilled.
- **Product Not Found**:If product_id is invalid (i.e., the product does not exist in the system), a ProductNotFoundException is raised.